



A Note on Message Ordering

credits for the slides to:
Timofei Istomin
Giuliano Mega
Luca Mottola

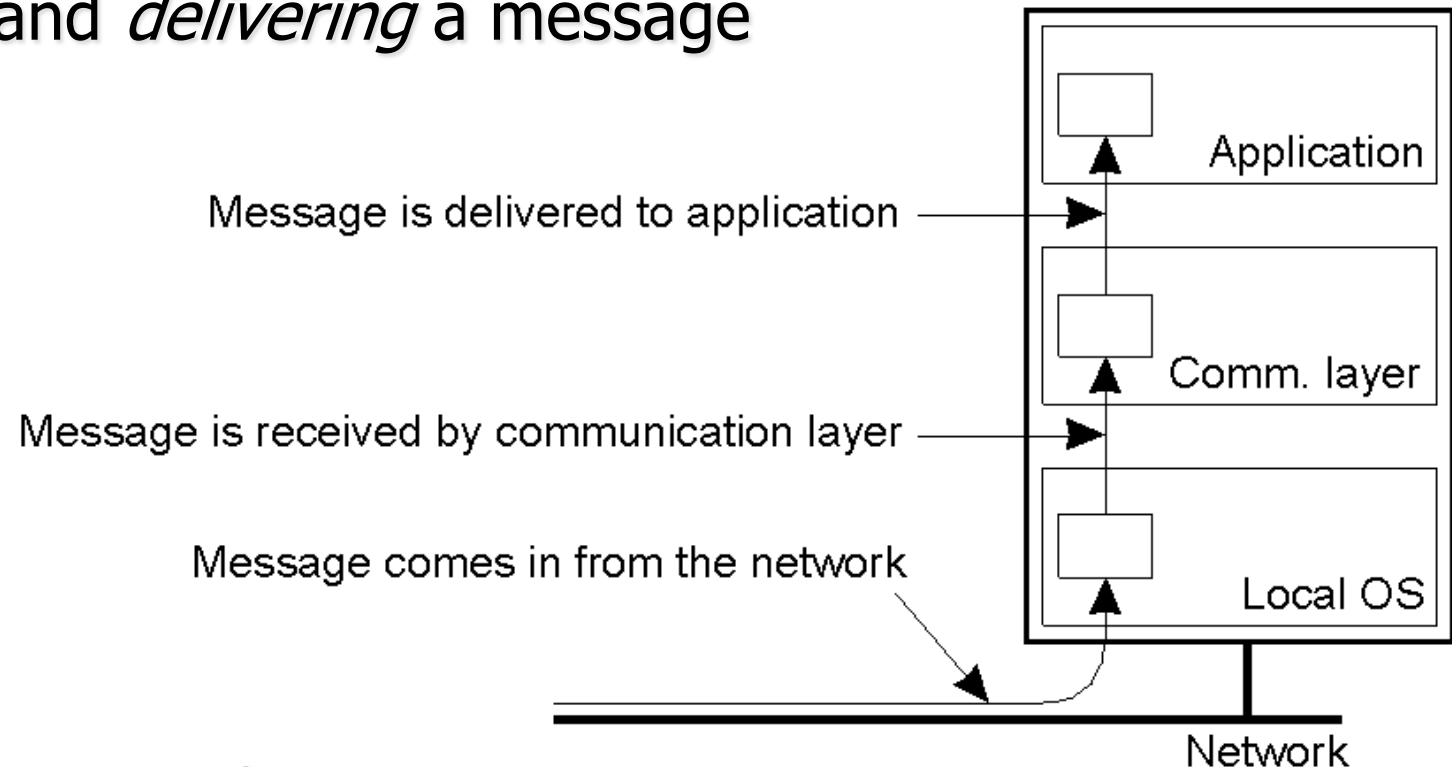
Happens-before Relation



- ◆ Let e_i and e_j be any two events in a distributed execution
- ◆ We say $e_i \rightarrow e_j$ if *any* of the following holds
 - ◆ e_i and e_j are two events on the same process and e_i comes before e_j
 - ◆ e_i is the sending event of message m , and e_j is the receive event for the same message m
 - ◆ there exist e_k such that $e_i \rightarrow e_k$ and $e_k \rightarrow e_j$
- ◆ If $e_i \rightarrow e_j$, e_j might have been caused by e_i

Receive vs. Deliver

- ◆ We make a distinction between *receiving* and *delivering* a message



- ◆ Already seen in TCP:
 - ◆ messages might be *received* out-of-order
 - ◆ but TCP buffers them and *delivers* them in order to the application

Message Ordering



◆ Let

- ◆ $S_p(m)$ be the send event for message m by process p
- ◆ $D_p(m)$ be the delivery event for message m by process p

◆ FIFO ordering

- ◆ $\forall p, q, m, m' : S_p(m) \rightarrow S_p(m') \Rightarrow D_q(m) \rightarrow D_q(m')$

◆ Causal ordering

- ◆ $\forall p, q, r, m, m' : S_p(m) \rightarrow S_q(m') \Rightarrow D_r(m) \rightarrow D_r(m')$

◆ Total ordering

- ◆ $\forall p, q, m, m' : D_p(m) \rightarrow D_p(m') \Leftrightarrow D_q(m) \rightarrow D_q(m')$

Senders might be different !

Only delivery matters
(not sending and not receiving)!

Message Ordering

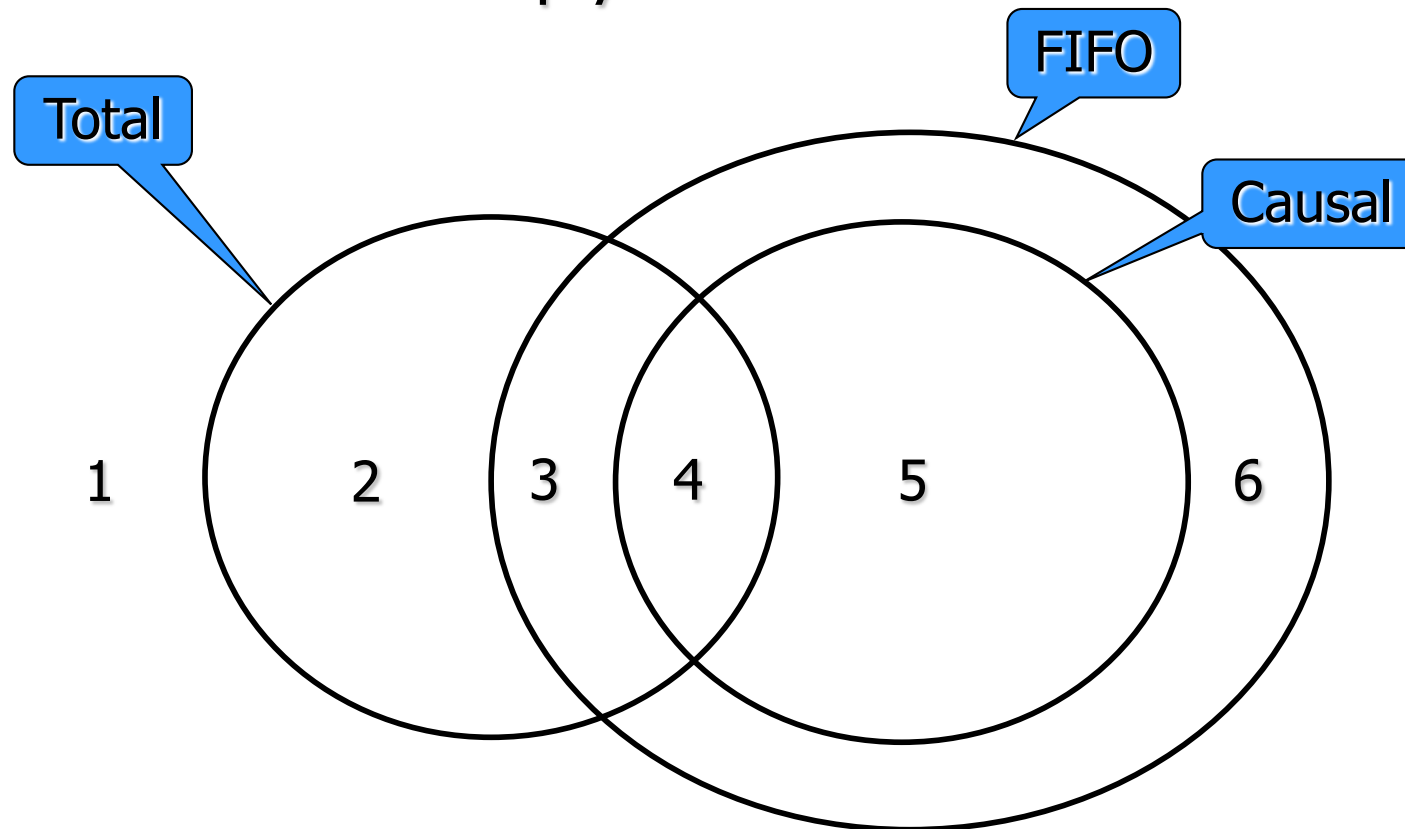


- ◆ Three base variants of a group communication service:
 - ◆ unordered
 - ◆ FIFO ordered
 - ◆ causally ordered
- ◆ ... where each of those can be:
 - ◆ totally ordered
 - ◆ not totally ordered
- ◆ ... for a total of six variants

Set Containments



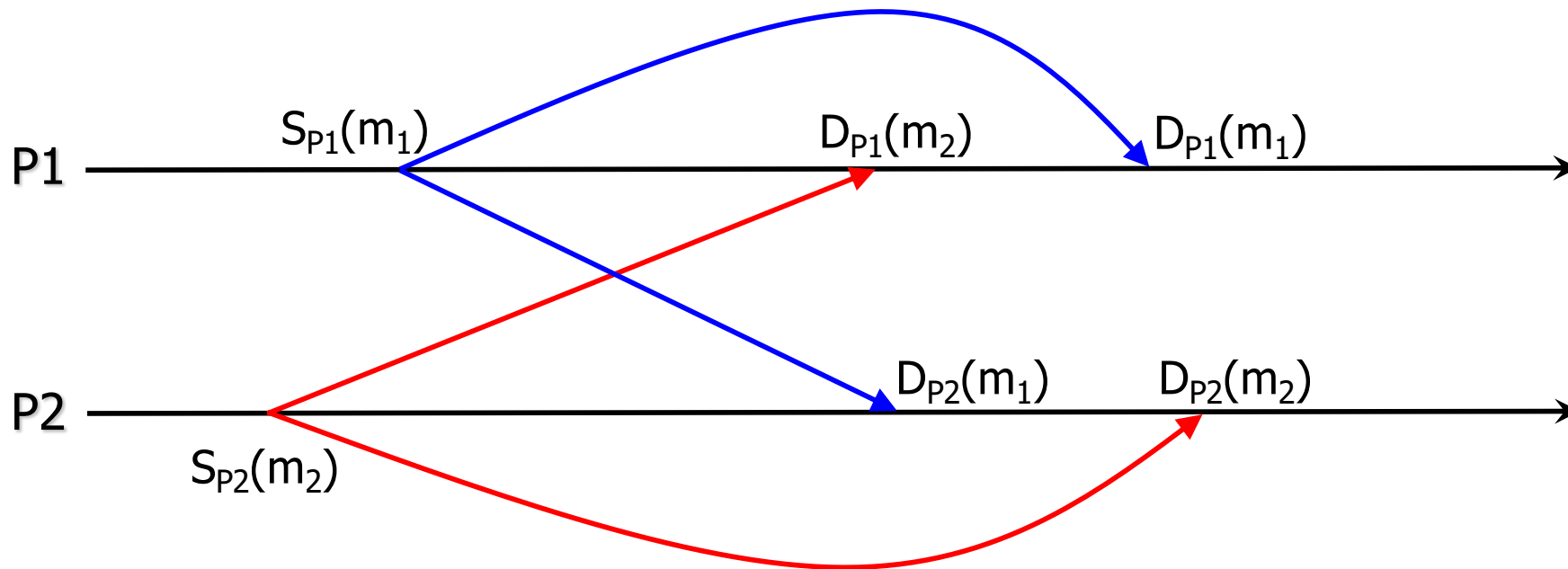
- ◆ Causal implies FIFO (thus, not FIFO implies not Causal)
- ◆ Total does NOT imply Causal
- ◆ Causal does NOT imply Total



Examples (1/5)



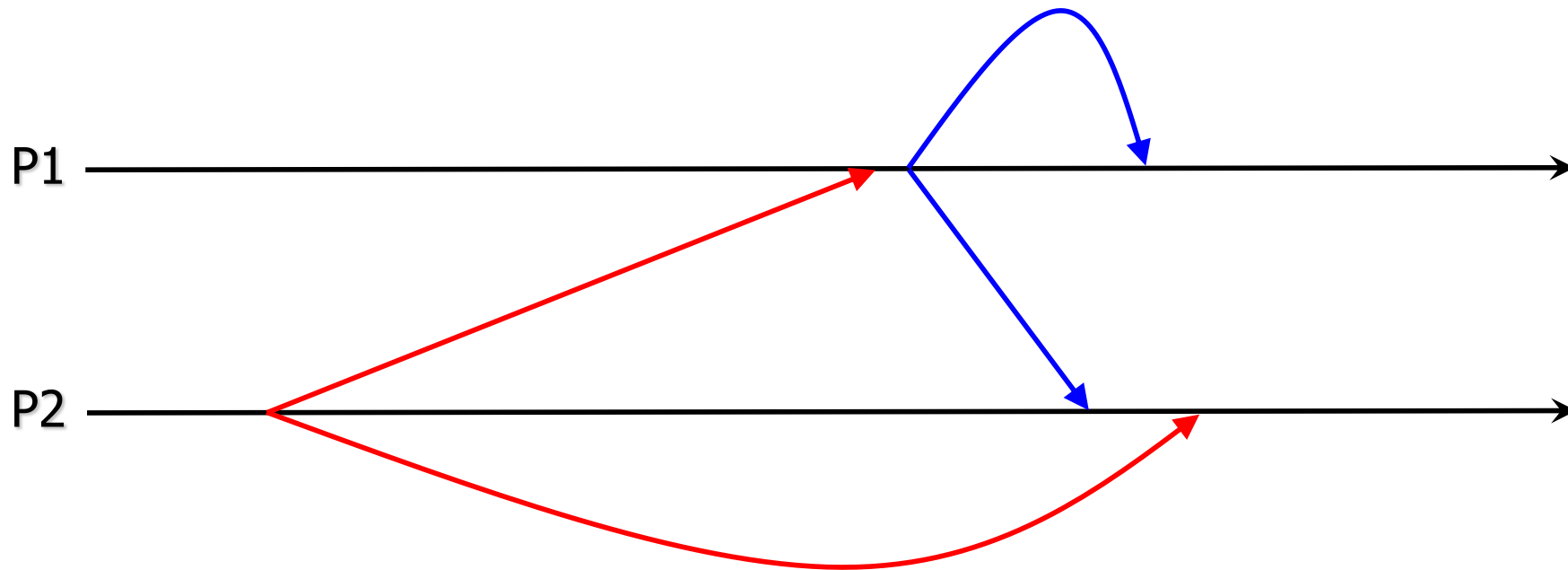
- ◆ FIFO
- ◆ Causal
- ◆ NOT Total



Examples (2/5)



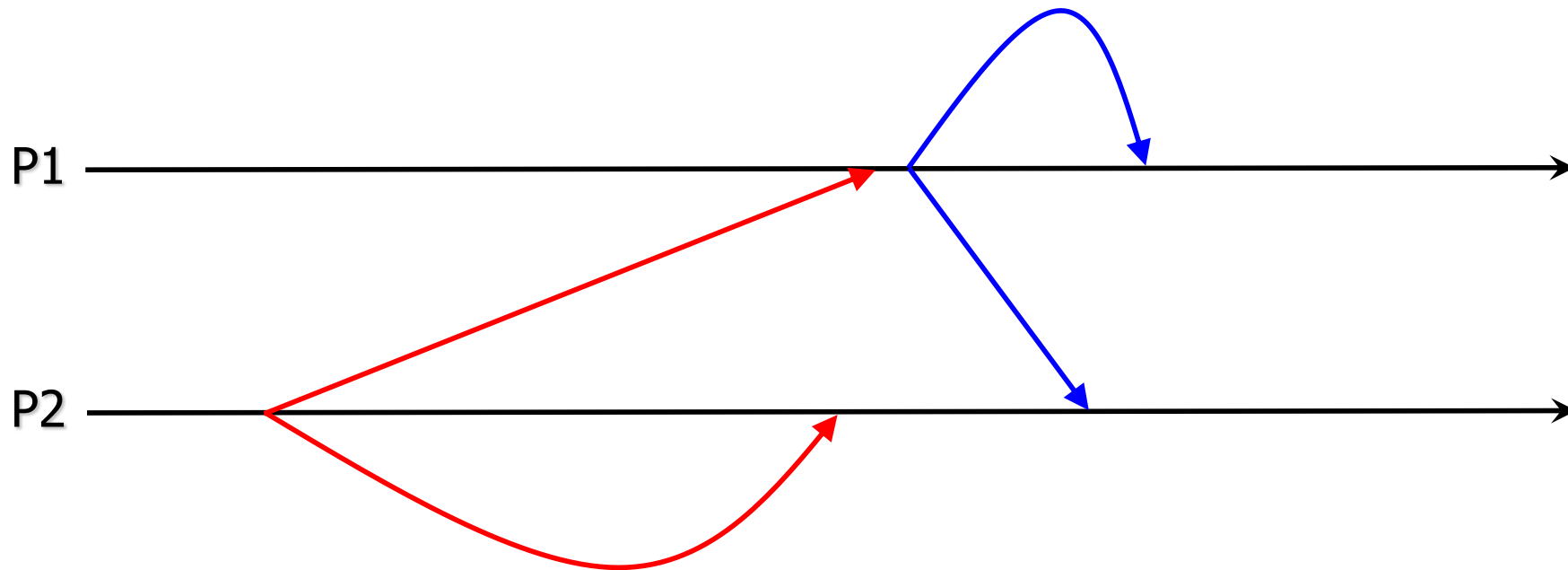
- ◆ FIFO
- ◆ NOT Causal
- ◆ NOT Total



Examples (3/5)



- ◆ FIFO
- ◆ Causal
- ◆ Total

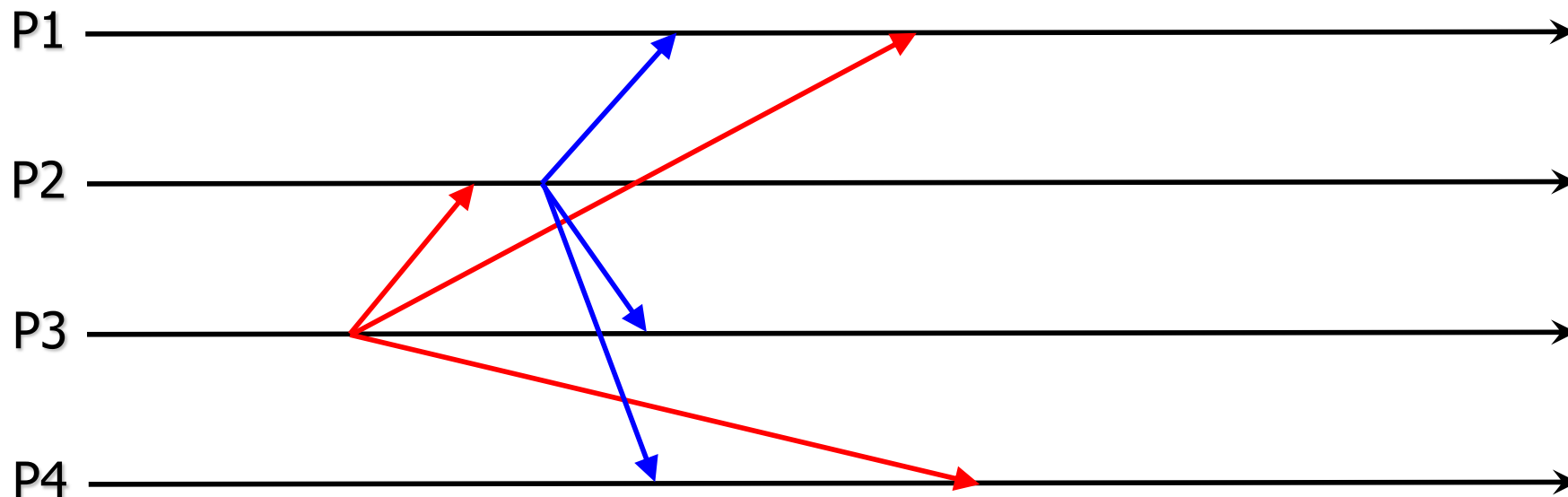


Examples (4/5)



- ◆ FIFO
- ◆ NOT Causal
- ◆ Total

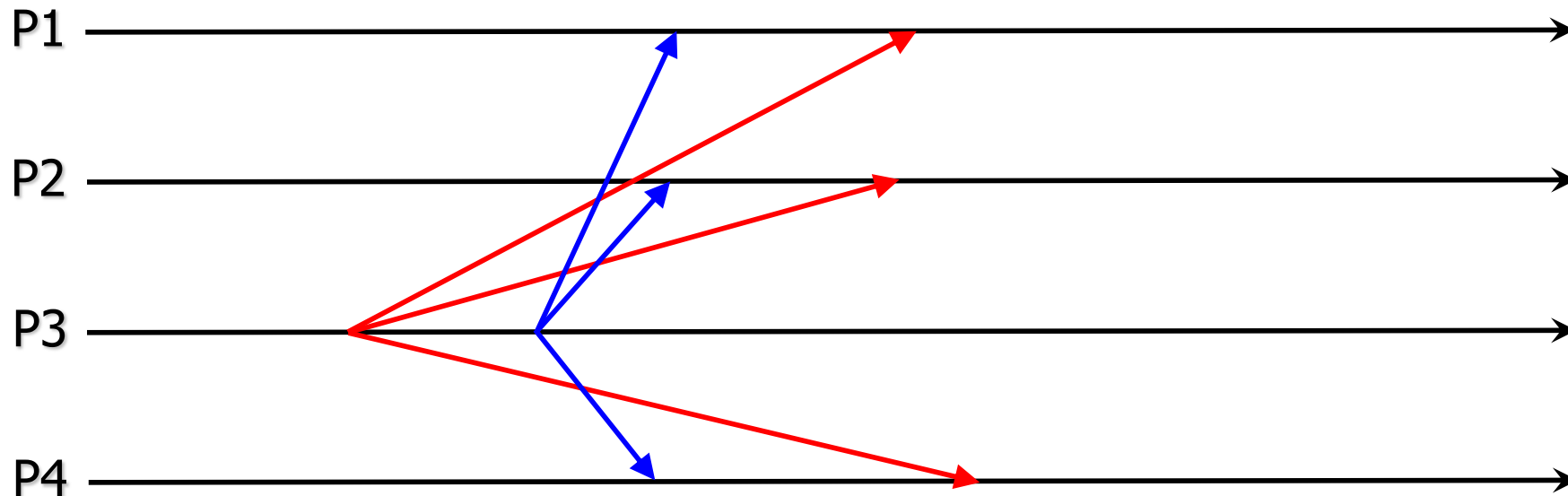
Note: In this case, senders do not deliver to themselves. This is a choice of the algorithm... (do you always send email also to yourself?)



Examples (5/5)



- ◆ NOT FIFO
- ◆ NOT Causal
- ◆ Total



Exercise



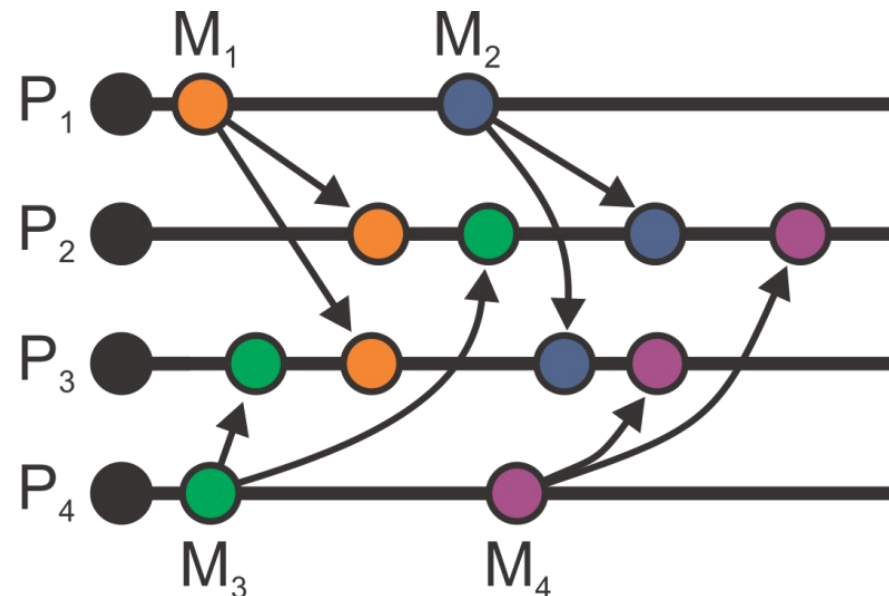
Process P1	Process P2	Process P3	Process P4
sends m1	receives m1	receives m3	sends m3
sends m2	receives m3	receives m1	sends m4
	receives m2	receives m2	
	receives m4	receives m4	

Is this total ordered? Causal? FIFO? Draw the time diagram!

Exercise

Process P1	Process P2	Process P3	Process P4
sends m1	receives m1	receives m3	sends m3
sends m2	receives m3	receives m1	sends m4
	receives m2	receives m2	
	receives m4	receives m4	

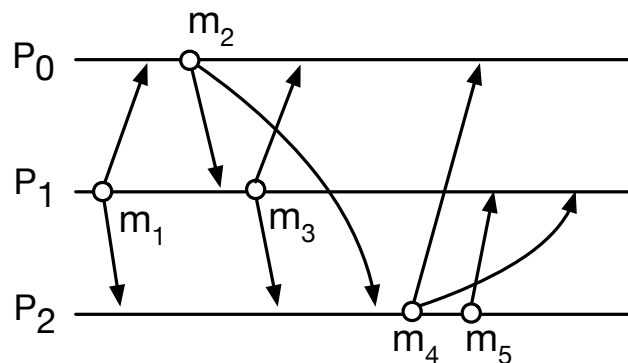
Is this total ordered? Causal? FIFO? Draw the time diagram!



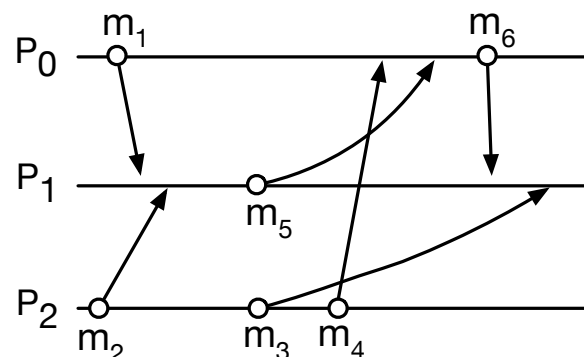
FIFO
Causal
NOT Totally ordered

Exercise

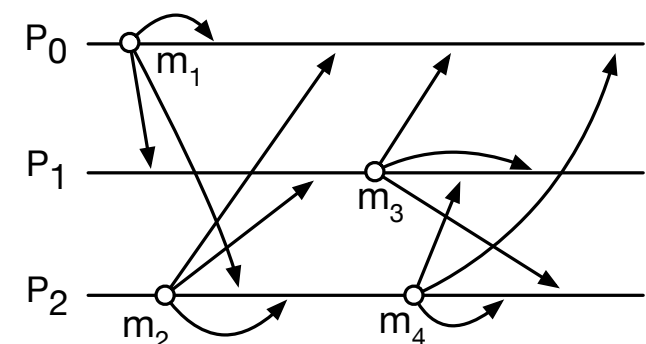
- ◆ In the context of message ordering schemes:
 1. provide the definitions for the three delivery schemes we have seen in class: *FIFO*, *causal*, and *total*. Briefly discuss the relationship between these schemes (e.g. are they independent? Does one imply the other?).
 2. For each of the executions depicted below, indicate whether or not they satisfy the delivery schemes of item (1). Should they not satisfy one or more of the schemes, indicate the first message(s) that causes *each scheme* to be violated, as well as the reason why they cause the violation.



(E1)



(E2)



(E3)